Bloque 3. Disparadores e integridad de la información

Los disparadores, también conocidos como **triggers**, permiten ejecutar automáticamente instrucciones SQL cuando se produce un determinado evento sobre una tabla. Son una herramienta importante para automatizar tareas, mantener la integridad de los datos y registrar los cambios realizados en una base de datos.

> **Objetivos del bloque**
>
> - Comprender qué es un disparador y cuándo se ejecuta.
> - Identificar los eventos que pueden activar un trigger.
> - Crear disparadores con las instrucciones `CREATE TRIGGER`.
> - Utilizar `OLD` y `NEW` para acceder a los datos afectados.
> - Automatizar validaciones, auditorías y operaciones relacionadas con la integridad.

1. Concepto de disparador (trigger)

Un **disparador** es un objeto almacenado en una base de datos que ejecuta automáticamente un conjunto de instrucciones SQL cuando ocurre un evento determinado sobre una tabla.

A diferencia de un procedimiento almacenado, que debe ejecutarse explícitamente mediante una llamada como `CALL`, un disparador se ejecuta de forma automática cuando se produce el evento para el que ha sido definido.

1.1. ¿Para qué se utilizan?

Los disparadores permiten automatizar diferentes tareas:

- **Validación de datos:** impedir que se almacenen valores incorrectos.
- **Auditoría:** registrar quién modifica los datos y cuándo.
- **Historial:** conservar los valores anteriores de los registros.
- **Actualización automática:** modificar información relacionada cuando cambia una tabla.
- **Integridad:** aplicar reglas adicionales que no se pueden expresar fácilmente mediante restricciones convencionales.

Ejemplo de funcionamiento
Imaginemos una tienda que almacena sus productos en una tabla `productos`. Cuando se modifica el precio de un producto, queremos guardar automáticamente el precio anterior y el nuevo precio en una tabla de historial.

El usuario únicamente ejecuta un `UPDATE`. El trigger se encarga de registrar el cambio sin que el usuario tenga que realizar ninguna operación adicional.

2. Eventos que activan los disparadores

En MariaDB, los disparadores se asocian a tres tipos de operaciones realizadas sobre una tabla:

2.1. INSERT

Se ejecuta cuando se inserta un nuevo registro en una tabla. Puede utilizarse para validar los datos antes de almacenarlos o registrar automáticamente la incorporación de nuevos registros.

```
INSERT INTO productos (nombre, precio, stock)
VALUES ('Teclado', 25.50, 10);
```

2.2. UPDATE

Se ejecuta cuando se modifica uno o varios registros existentes. Resulta especialmente útil para controlar cambios, mantener historiales o comprobar que los nuevos valores cumplen determinadas condiciones.

```
UPDATE productos
SET precio = 29.90
WHERE id = 1;
```

2.3. DELETE

Se ejecuta cuando se elimina uno o varios registros. Puede utilizarse para guardar una copia de la información eliminada antes de que desaparezca de la tabla original.

```
DELETE FROM productos
WHERE id = 1;
```

> **Importante**
> Los triggers se ejecutan por cada fila afectada por la operación. Si una instrucción `UPDATE` modifica diez registros, el disparador asociado se ejecutará diez veces, una por cada fila.

3. Momentos de ejecución: BEFORE y AFTER

Además del evento que activa el disparador, es necesario indicar cuándo debe ejecutarse respecto a la operación que lo provoca.

3.1. BEFORE

El disparador se ejecuta **antes** de realizar la operación sobre la tabla.

Se utiliza principalmente para:

- Validar los valores que se van a insertar o modificar.
- Rechazar operaciones que incumplen determinadas reglas.
- Modificar valores antes de almacenarlos.
- Aplicar reglas de negocio adicionales.

3.2. AFTER

El disparador se ejecuta **después** de realizar la operación sobre la tabla.

Se utiliza principalmente para:

- Registrar modificaciones en tablas de auditoría.
- Guardar información histórica.
- Actualizar información complementaria.
- Realizar operaciones relacionadas con el cambio producido.

3.3. Combinaciones posibles

Momento	Evento	Uso habitual
BEFORE	INSERT	Validar datos antes de insertar.
AFTER	INSERT	Registrar nuevas incorporaciones.
BEFORE	UPDATE	Comprobar o ajustar nuevos valores.
AFTER	UPDATE	Guardar los cambios realizados.
BEFORE	DELETE	Comprobar o conservar información antes de eliminar.
AFTER	DELETE	Registrar la eliminación realizada.

4. Creación de disparadores con CREATE TRIGGER

Para crear un disparador en MariaDB se utiliza la instrucción `CREATE TRIGGER`.

4.1. Sintaxis general

```
DELIMITER //

CREATE TRIGGER nombre_trigger
BEFORE | AFTER INSERT | UPDATE | DELETE
ON nombre_tabla
FOR EACH ROW
BEGIN
    -- Instrucciones SQL
END//

DELIMITER ;
```

Los elementos principales son:

- `CREATE TRIGGER`: crea el disparador.
- `nombre_trigger`: identifica el disparador.
- `BEFORE` o `AFTER`: establece el momento de ejecución.
- `INSERT`, `UPDATE` o `DELETE`: indica el evento que lo activa.

- `ON` : especifica la tabla asociada.
- `FOR EACH ROW` : indica que se ejecutará para cada fila afectada.
- `BEGIN ... END` : contiene las instrucciones que ejecutará el disparador.

> **Uso de DELIMITER**
> Cuando se crean procedimientos o triggers con varias instrucciones, es necesario cambiar temporalmente el delimitador en la consola o en el editor SQL de Workbench. Así se evita que el punto y coma de las instrucciones internas finalice prematuramente la definición.
>
> `DELIMITER` es una instrucción del cliente SQL, no una instrucción que forme parte del cuerpo del trigger almacenado.

4.2. Consultar los disparadores existentes

Para consultar los disparadores de una base de datos:

```
SHOW TRIGGERS;
```

También se puede consultar la definición de un trigger concreto:

```
SHOW CREATE TRIGGER nombre_trigger;
```

4.3. Eliminar un disparador

Para eliminar un trigger se utiliza:

```
DROP TRIGGER IF EXISTS nombre_trigger;
```

5. Variables OLD y NEW

Los disparadores pueden necesitar consultar los valores de la fila sobre la que se está realizando la operación. Para ello, MariaDB proporciona dos referencias especiales: `OLD` y `NEW`.

5.1. NEW

`NEW` permite acceder a los valores nuevos de una fila. Se utiliza en operaciones de inserción y modificación.

Por ejemplo:

```
NEW.precio
NEW.nombre
NEW.stock
```

5.2. OLD

`OLD` permite acceder a los valores que tenía la fila antes de realizar una modificación o eliminación.

Por ejemplo:

```
OLD.precio
OLD.nombre
OLD.stock
```

5.3. Disponibilidad según el evento

Evento	OLD	NEW
INSERT	No disponible	Disponible
UPDATE	Disponible	Disponible
DELETE	Disponible	No disponible

> **Regla para recordar**
> En una inserción solo existe `NEW`, porque el registro todavía no tenía valores anteriores. En una eliminación solo existe `OLD`, porque el registro va a desaparecer. En una modificación existen ambos.

5.4. Modificar valores mediante NEW

En un trigger `BEFORE INSERT` o `BEFORE UPDATE` se pueden modificar determinados valores de `NEW` para ajustar la información antes de almacenarla.

Por ejemplo, podemos convertir automáticamente el nombre de un producto a mayúsculas:

```
DELIMITER //

CREATE TRIGGER trg_productos_mayusculas
BEFORE INSERT ON productos
FOR EACH ROW
BEGIN
    SET NEW.nombre = UPPER(NEW.nombre);
END//

DELIMITER ;
```

Si se inserta un producto con el nombre `teclado`, el valor almacenado será `TECLADO`.

6. Automatización de tareas relacionadas con la integridad

La integridad de la información consiste en garantizar que los datos almacenados sean correctos, coherentes y cumplan las reglas establecidas para la aplicación.

MariaDB dispone de diferentes mecanismos para mantener la integridad:

- **Restricciones:** `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL` y `CHECK`.
- **Tipos de datos:** limitan los valores que pueden almacenarse.
- **Transacciones:** permiten confirmar o deshacer operaciones relacionadas.
- **Triggers:** automatizan comprobaciones y operaciones adicionales.

> **Buena práctica**
> Siempre que sea posible, las reglas de integridad deben definirse mediante restricciones propias del modelo relacional. Los triggers deben utilizarse para aquellas validaciones o automatizaciones que requieren lógica adicional.

7. Ejemplo práctico: validación de precios

Vamos a crear una tabla de productos y un disparador que impida insertar o modificar productos con un precio igual o inferior a cero.

7.1. Crear la tabla

```
CREATE DATABASE IF NOT EXISTS tienda;
USE tienda;

CREATE TABLE productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0
);
```

7.2. Crear el trigger BEFORE INSERT

```
DELIMITER //

CREATE TRIGGER trg_validar_precio_insert
BEFORE INSERT ON productos
FOR EACH ROW
BEGIN
    IF NEW.precio <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El precio debe ser superior a cero';
    END IF;
END//

DELIMITER ;
```

La instrucción `SIGNAL` permite generar un error desde el disparador. En este caso, se utiliza para impedir que se realice una inserción que incumple la regla definida.

7.3. Probar el funcionamiento

Una inserción correcta:

```
INSERT INTO productos (nombre, precio, stock)
VALUES ('Monitor', 149.99, 15);
```

Una inserción incorrecta:

```
INSERT INTO productos (nombre, precio, stock)
VALUES ('Ratón', -5, 20);
```

En el segundo caso, el trigger genera un error y la fila no se inserta.

7.4. Validar también las modificaciones

La regla anterior solo controla las inserciones. Si queremos evitar que posteriormente se establezca un precio incorrecto, debemos crear otro disparador para el evento `UPDATE`.

```
DELIMITER //

CREATE TRIGGER trg_validar_precio_update
BEFORE UPDATE ON productos
FOR EACH ROW
BEGIN
    IF NEW.precio <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El precio debe ser superior a cero';
    END IF;
END//

DELIMITER ;
```

Podemos comprobarlo mediante:

```
UPDATE productos
SET precio = -10
WHERE id = 1;
```

La operación será rechazada porque el nuevo precio no cumple la condición establecida.

8. Auditoría e historial de modificaciones

Una de las aplicaciones más importantes de los triggers es registrar los cambios realizados en una base de datos. Esto permite conocer qué información se ha modificado y conservar los valores anteriores.

8.1. Crear una tabla de historial

Vamos a crear una tabla que almacene los cambios de precio realizados sobre los productos.

```
CREATE TABLE historial_precios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto_id INT NOT NULL,
    precio_anterior DECIMAL(10,2),
    precio_nuevo DECIMAL(10,2),
    fecha_cambio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    usuario VARCHAR(100)
);
```

Esta tabla no tiene una clave foránea hacia `productos` porque puede interesarnos conservar el historial incluso después de eliminar el producto original.

8.2. Crear el trigger AFTER UPDATE

Cuando se modifique el precio de un producto, queremos insertar automáticamente un registro en la tabla de historial.

```
DELIMITER //

CREATE TRIGGER trg_historial_precios
AFTER UPDATE ON productos
FOR EACH ROW
BEGIN
    IF OLD.precio <> NEW.precio THEN

        INSERT INTO historial_precios (
            producto_id,
            precio_anterior,
            precio_nuevo,
            usuario
        )
        VALUES (
            OLD.id,
            OLD.precio,
            NEW.precio,
            CURRENT_USER()
        );

    END IF;
END//

DELIMITER ;
```

En este ejemplo:

- `OLD.id` identifica el producto modificado.
- `OLD.precio` contiene el precio anterior.
- `NEW.precio` contiene el precio nuevo.
- `CURRENT_USER()` devuelve la cuenta MariaDB utilizada para determinar los privilegios de la conexión.

8.3. Comprobar el historial

Modificamos el precio de un producto:

```
UPDATE productos
SET precio = 159.99
WHERE id = 1;
```

Consultamos el historial:

```
SELECT *
FROM historial_precios
ORDER BY fecha_cambio DESC;
```

Se habrá creado automáticamente un registro con el precio anterior y el precio nuevo.

> **Consideración sobre la auditoría**
> `CURRENT_USER()` identifica la cuenta MariaDB que se utiliza para comprobar los privilegios de la conexión. No siempre coincide con la identidad de la persona física que realiza la operación. Si la aplicación necesita identificar al usuario final, deberá proporcionar esa información mediante un mecanismo controlado.

9. Registrar eliminaciones

También podemos conservar los datos de un producto antes de que sea eliminado. Para ello, utilizaremos un trigger `BEFORE DELETE` o `AFTER DELETE`.

Primero creamos una tabla para conservar los productos eliminados:

```
CREATE TABLE productos_eliminados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto_id INT,
    nombre VARCHAR(100),
    precio DECIMAL(10,2),
    stock INT,
    fecha_eliminacion DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Después creamos el disparador:

```
DELIMITER //

CREATE TRIGGER trg_productos_eliminados
AFTER DELETE ON productos
FOR EACH ROW
BEGIN
    INSERT INTO productos_eliminados (
        producto_id,
        nombre,
        precio,
        stock
    )
    VALUES (
        OLD.id,
        OLD.nombre,
        OLD.precio,
        OLD.stock
    );
END//

DELIMITER ;
```

Ahora, cuando se elimine un producto, sus datos quedarán registrados en la tabla `productos_eliminados`.

```
DELETE FROM productos
WHERE id = 1;

SELECT *
FROM productos_eliminados;
```

10. Integridad y consistencia: aspectos importantes

10.1. Los triggers forman parte de la operación original

Las instrucciones ejecutadas por un trigger forman parte de la operación que lo ha activado. Si el disparador genera un error, la operación original puede ser rechazada.

Para que las operaciones relacionadas puedan beneficiarse de la transaccionalidad, las tablas implicadas deben utilizar un motor transaccional, como **InnoDB**.

10.2. Evitar modificaciones recursivas

Un trigger no debe intentar modificar directamente la misma tabla que ha provocado su ejecución. MariaDB restringe este tipo de operaciones para evitar comportamientos recursivos y problemas de consistencia.

Por ejemplo, un trigger asociado a `productos` no debe ejecutar un `UPDATE productos` sobre esa misma tabla. Para ajustar los valores de la fila afectada se utiliza `NEW` en los triggers `BEFORE`, cuando la operación lo permite.

10.3. No sustituir las restricciones por triggers

Antes de crear un disparador para controlar una regla, debemos comprobar si puede resolverse mediante una restricción estándar.

Necesidad	Mecanismo recomendado
Evitar valores nulos	NOT NULL
Evitar identificadores duplicados	PRIMARY KEY o UNIQUE
Relacionar registros entre tablas	FOREIGN KEY
Controlar una condición sobre una columna	CHECK
Registrar automáticamente modificaciones	TRIGGER
Aplicar reglas de negocio complejas	TRIGGER o procedimiento almacenado, según el caso

11. Gestión y buenas prácticas con triggers

Antes de incorporar disparadores a una base de datos, conviene tener en cuenta algunas recomendaciones:

- Utilizar nombres descriptivos, por ejemplo `trg_productos_auditoria`.
- Realizar únicamente las operaciones necesarias dentro del trigger.
- Evitar consultas y operaciones innecesariamente complejas.
- Documentar las reglas de negocio implementadas.
- Comprobar los efectos de los triggers antes de desplegar cambios.
- Revisar los triggers existentes cuando se modifica la estructura de una tabla.
- Evitar que varios triggers realicen operaciones contradictorias.
- Probar tanto las operaciones correctas como los casos que deben generar errores.

11.1. Consultar la definición de un trigger

```
SHOW CREATE TRIGGER trg_historial_precios;
```

11.2. Consultar los triggers de una base de datos

```
USE tienda;

SHOW TRIGGERS;
```

11.3. Eliminar un trigger

```
DROP TRIGGER IF EXISTS trg_historial_precios;
```

12. Resumen del bloque

Concepto	Descripción
Trigger	Objeto que ejecuta instrucciones automáticamente ante un evento.
INSERT	Activa el trigger al insertar registros.
UPDATE	Activa el trigger al modificar registros.
DELETE	Activa el trigger al eliminar registros.
BEFORE	Ejecuta el trigger antes de la operación.
AFTER	Ejecuta el trigger después de la operación.
NEW	Permite consultar los valores nuevos de una fila.
OLD	Permite consultar los valores anteriores de una fila.
SIGNAL	Genera un error desde el trigger para rechazar una operación.
SHOW TRIGGERS	Consulta los disparadores existentes.
DROP TRIGGER	Elimina un disparador.

13. Ejercicios prácticos de comprobación

Los siguientes ejercicios permiten comprobar los conceptos trabajados en el bloque utilizando la base de datos `tienda`.

Ejercicio 1. Validación de datos

Crea un disparador que impida insertar productos cuyo stock sea negativo. Comprueba su funcionamiento con una inserción correcta y otra incorrecta.

Ejercicio 2. Control de modificaciones

Crea un trigger que impida modificar el precio de un producto si el nuevo precio es inferior al anterior. Comprueba que una modificación que aumenta el precio funciona correctamente.

Ejercicio 3. Historial de cambios

Amplía la tabla de historial para registrar también los cambios realizados sobre el stock de los productos. El registro deberá almacenar el valor anterior, el nuevo valor y la fecha del cambio.

Ejercicio 4. Registro de eliminaciones

Crea un trigger que registre en una tabla independiente los datos de los productos eliminados. Comprueba que la información se conserva después de ejecutar un `DELETE`.

Ejercicio 5. Comprobación de automatización

Realiza varias operaciones de inserción, modificación y eliminación. Consulta las tablas de auditoría y explica qué operaciones han provocado la ejecución de cada disparador.

> **Conclusión**
> Los triggers permiten incorporar automatismos directamente en la base de datos. Su uso facilita la validación de información, el registro de modificaciones y la aplicación de reglas de negocio. Sin embargo, deben diseñarse cuidadosamente para evitar efectos inesperados y mantener la coherencia de los datos.